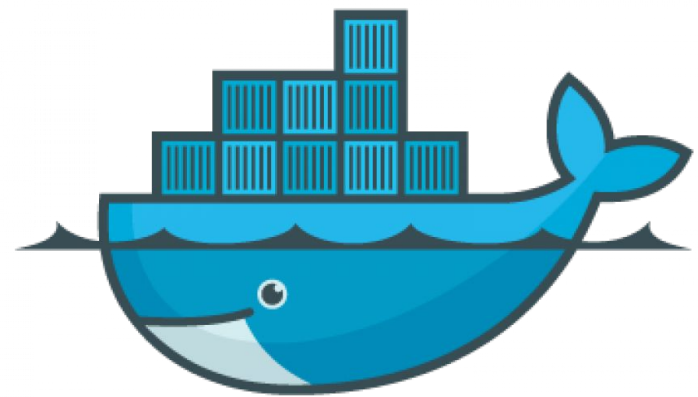


Docker e aplicações Docker

MCS. JOSE ALFREDO MENDOZA PEÑALOZA

Docker

Docker é uma plataforma de virtualização baseada em contêineres que permite desenvolver, empacotar, distribuir e executar aplicações de forma padronizada e isolada. Seu principal objetivo é garantir que uma aplicação funcione da mesma maneira em diferentes ambientes computacionais, independentemente do sistema operacional, da infraestrutura ou das dependências instaladas.



docker

Docker

Diferentemente das máquinas virtuais tradicionais, que virtualizam o hardware completo e executam um sistema operacional próprio, o Docker utiliza contêineres que compartilham o núcleo (kernel) do sistema operacional hospedeiro, tornando a execução mais leve, rápida e eficiente.

O Docker foi criado para resolver problemas relacionados à compatibilidade de software, gerenciamento de dependências, portabilidade de aplicações e implantação em diferentes ambientes, como desenvolvimento, testes, homologação e produção.

Framework

Um framework é uma estrutura de software composta por bibliotecas, ferramentas, componentes e regras de desenvolvimento que fornecem uma base pronta para a criação de aplicações. Seu objetivo é facilitar o desenvolvimento de sistemas, reduzindo a necessidade de criar funcionalidades básicas do zero. O framework estabelece uma arquitetura e um conjunto de boas práticas que orientam o programador durante o desenvolvimento. Exemplos de frameworks amplamente utilizados incluem Flask e Django para Python, Spring Boot para Java e Angular para aplicações web. Ao utilizar um framework, os desenvolvedores podem concentrar seus esforços na implementação das funcionalidades específicas do projeto, aumentando a produtividade e a padronização do código.

Contêiner (Container)

Um contêiner é uma unidade de software leve e portátil que contém tudo o que uma aplicação necessita para ser executada:

- Código da aplicação;
- Bibliotecas;
- Dependências;
- Arquivos de configuração;
- Variáveis de ambiente;
- Ferramentas necessárias para execução.

O contêiner é executado de forma isolada do sistema operacional hospedeiro e dos demais contêineres.

Por exemplo, um servidor web Apache pode ser executado dentro de um contêiner Docker sem necessidade de instalação direta no sistema operacional principal.

Imagem (Image)

Uma imagem Docker é um modelo imutável utilizado para criar contêineres.

A imagem contém:

- Sistema operacional básico;
- Bibliotecas;
- Dependências;
- Aplicação;
- Configurações.

Pode-se considerar uma imagem como um "molde" a partir do qual diversos contêineres podem ser criados.

Imagem (Image)

Exemplo:

Uma imagem Python pode conter:

- Ubuntu Linux;
- Python 3.12;
- Bibliotecas Flask;
- Código da aplicação.

A partir dessa imagem, podem ser criados múltiplos contêineres idênticos

Dockerfile

Dockerfile é um arquivo de texto contendo instruções para a construção automática de uma imagem Docker.

Exemplo

Neste exemplo:

- FROM define a imagem base;
- WORKDIR define o diretório de trabalho;
- COPY copia arquivos para a imagem;
- RUN executa comandos durante a construção;
- CMD define o comando inicial do contêiner.

```
</> dockerfile

FROM python:3.12

WORKDIR /app

COPY . .

RUN pip install flask

CMD ["python", "app.py"]
```


Docker Engine

O Docker Engine é o componente principal da plataforma Docker e pode ser considerado o núcleo responsável pela criação, execução e gerenciamento de contêineres. Trata-se de uma aplicação cliente-servidor que permite a comunicação entre o usuário e os recursos de containerização do sistema. O Docker Engine gerencia imagens, contêineres, redes e volumes, fornecendo uma infraestrutura completa para a execução de aplicações isoladas. Ele é composto pelo Docker Daemon, pela Docker API e pela Docker CLI, que trabalham em conjunto para permitir a administração eficiente dos contêineres.

Docker Daemon

O Docker Daemon, também conhecido como "dockerd", é um serviço executado em segundo plano no sistema operacional hospedeiro. Sua principal função é gerenciar os objetos Docker, incluindo imagens, contêineres, redes e volumes. Quando um usuário executa um comando Docker, a solicitação é enviada ao Daemon, que interpreta as instruções e realiza as operações necessárias. O Docker Daemon também é responsável por baixar imagens de registros remotos, criar novos contêineres, monitorar sua execução e liberar recursos quando eles são encerrados. Em sistemas Linux, normalmente é executado como um serviço do sistema e permanece ativo continuamente.

Docker Daemon

Processo executado em segundo plano responsável por:

- Criar contêineres;
- Executar contêineres;
- Gerenciar imagens;
- Gerenciar redes;
- Gerenciar volumes.

Docker CLI

A Docker CLI (Command Line Interface) é a interface de linha de comando utilizada pelos usuários para interagir com o Docker. Ela fornece um conjunto de comandos que permitem criar, executar, parar, remover e monitorar contêineres. Quando um comando é digitado no terminal, a Docker CLI envia a solicitação ao Docker Daemon por meio da Docker API. A CLI simplifica a administração dos recursos Docker e constitui a principal ferramenta utilizada por desenvolvedores e administradores de sistemas para controlar ambientes containerizados.

Docker CLI

Interface de linha de comando utilizada pelo usuário.

```
docker run nginx  
docker ps  
docker images  
docker stop container_id
```

Docker API

A Docker API é uma interface de programação que permite a comunicação entre aplicações externas e o Docker Engine. Ela utiliza requisições baseadas em protocolos de rede para enviar comandos e receber informações sobre o estado dos contêineres, imagens, redes e volumes. A Docker CLI utiliza a Docker API para transmitir as solicitações dos usuários ao Docker Daemon. Além disso, ferramentas de automação, plataformas de gerenciamento de infraestrutura e aplicações de terceiros podem utilizar a API para controlar o Docker de forma programática, sem necessidade de interação manual.

Docker Hub

Docker Hub é um repositório online de imagens Docker.

Funciona de maneira semelhante ao GitHub, mas destinado ao armazenamento de imagens.

Exemplos de imagens disponíveis:

- Ubuntu;
- Debian;
- Nginx;
- Apache;
- MySQL;
- PostgreSQL;
- Redis;
- Python;
- Node.js.

Etapa 1 – Criação da Imagem

O desenvolvedor cria um Dockerfile contendo todas as instruções necessárias para o ambiente da aplicação.

```
docker build -t minha_aplicacao .
```

O Docker gera uma imagem contendo:

- Sistema operacional base;
- Dependências;
- Aplicação.

Etapa 2 – Armazenamento da Imagem

A imagem pode ser armazenada:

- Localmente;
- Docker Hub;
- Registro privado.

Exemplo:

```
docker push usuario/minha_aplicacao
```

Etapa 3 – Criação do Container

A partir da imagem, cria-se um container.

```
docker run minha_aplicacao
```

Neste momento:

- Docker lê a imagem.
- Cria uma camada gravável.
- Inicializa os processos definidos.
- Executa a aplicação.

Etapa 4 – Execução

O contêiner passa a executar a aplicação de forma isolada.

```
docker run -p 5000:5000 minha_aplicacao
```

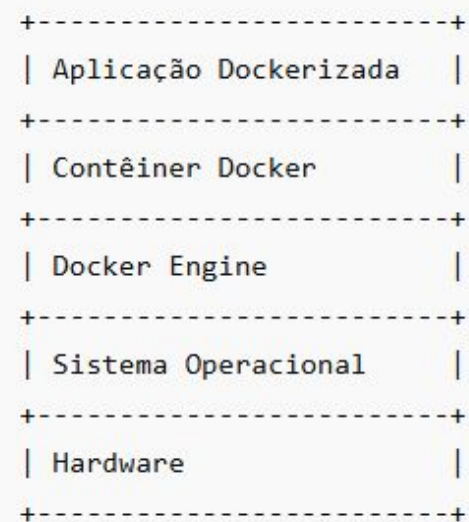
O parâmetro:

```
-p 5000:5000
```

mapeia a porta do contêiner para o sistema hospedeiro.

Arquitetura do Docker

A arquitetura do Docker é composta por diversos componentes que trabalham em conjunto para fornecer um ambiente de execução de contêineres. Na camada superior encontram-se as aplicações executadas dentro dos contêineres. Esses contêineres são gerenciados pelo Docker Engine, que atua como intermediário entre as aplicações e o sistema operacional hospedeiro. O Docker Engine comunica-se com o kernel do sistema operacional para criar processos isolados, gerenciar recursos computacionais e controlar o acesso à rede e ao armazenamento. Além disso, a arquitetura inclui repositórios de imagens, como o Docker Hub, que permitem armazenar e distribuir imagens para diferentes ambientes. Essa estrutura proporciona portabilidade, escalabilidade e facilidade de implantação das ap



Kernel

O kernel é o núcleo do sistema operacional e atua como intermediário entre o hardware e os programas executados no computador. Sua função é gerenciar recursos como processador, memória, armazenamento e dispositivos de entrada e saída, garantindo que as aplicações utilizem esses recursos de maneira segura e eficiente. No contexto do Docker, o kernel é fundamental porque os contêineres compartilham o mesmo núcleo do sistema operacional hospedeiro, utilizando mecanismos de isolamento e controle de recursos para executar aplicações de forma leve e eficiente.

Como o Isolamento é Realizado

O isolamento no Docker é obtido por meio de recursos nativos do kernel Linux, principalmente Namespaces e Control Groups (cgroups). Os Namespaces criam ambientes independentes para processos, sistemas de arquivos, interfaces de rede, usuários e identificadores de processos, permitindo que cada contêiner possua sua própria visão do sistema. Já os cgroups controlam a quantidade de recursos que cada contêiner pode utilizar, como memória RAM, processador, armazenamento e largura de banda de rede. Dessa forma, múltiplos contêineres podem ser executados simultaneamente no mesmo sistema operacional sem interferirem diretamente uns nos outros, garantindo maior segurança, estabilidade e eficiência no uso dos recursos.

Namespaces

Fornecem isolamento para:

- Processos;
- Usuários;
- Rede;
- Sistema de arquivos;
- Hostname.

Cada contêiner possui sua própria visão do sistema.

Control Groups (cgroups)

Permitem controlar recursos como:

- CPU;
- Memória RAM;
- Disco;
- Rede.

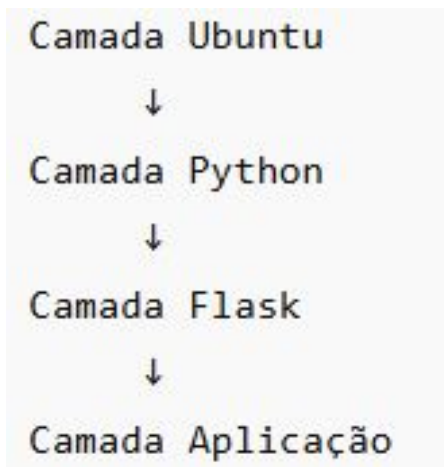
```
docker run --memory=512m nginx
```

O contêiner utilizará no máximo 512 MB de memória.

Union File System

Sistema de arquivos baseado em camadas.

Cada imagem é composta por múltiplas camadas reutilizáveis.



Isso reduz significativamente o espaço em disco.

Microserviços

Microserviços são uma arquitetura de software que divide uma aplicação em diversos serviços independentes e especializados, cada um responsável por uma funcionalidade específica. Esses serviços podem ser desenvolvidos, implantados e escalados separadamente, comunicando-se por meio de APIs ou protocolos de rede. Essa abordagem facilita a manutenção, aumenta a flexibilidade do sistema e permite que diferentes partes da aplicação evoluam de forma independente, sendo amplamente utilizada em ambientes de computação em nuvem e sistemas distribuídos.

Redes Docker

As redes Docker são mecanismos que permitem a comunicação entre contêineres, aplicações externas e o sistema hospedeiro. O Docker oferece diferentes modos de rede para atender a diversos cenários de utilização. A rede Bridge é o modo padrão e permite a comunicação entre contêineres localizados na mesma máquina. A rede Host faz com que o contêiner utilize diretamente a pilha de rede do sistema operacional hospedeiro, eliminando o isolamento de rede. A rede None desativa completamente o acesso à rede para o contêiner. Já a rede Overlay possibilita a comunicação entre contêineres distribuídos em diferentes servidores, sendo amplamente utilizada em ambientes de cluster e orquestração. O gerenciamento de redes é fundamental para o funcionamento de aplicações distribuídas e arquiteturas baseadas em microsserviços.

Redes no Docker

Docker oferece diferentes tipos de redes.

- Bridge:

Rede padrão utilizada para comunicação entre contêineres em uma mesma máquina.

```
docker network create minha_rede
```

- Host:

O contêiner compartilha a pilha de rede do hospedeiro.

```
docker run --network host nginx
```

Redes no Docker

- None

O contêiner é executado sem acesso à rede.

```
docker run --network none nginx
```

- Overlay

Permite comunicação entre contêineres distribuídos em diferentes servidores.

Muito utilizada em ambientes Docker Swarm e clusters.

Volumes Docker

Os volumes Docker são mecanismos de armazenamento persistente utilizados para preservar dados gerados pelos contêineres. Por padrão, os dados armazenados dentro de um contêiner são removidos quando ele é excluído. Os volumes resolvem esse problema ao armazenar os dados em uma área independente do ciclo de vida do contêiner. Dessa forma, informações importantes, como bancos de dados, arquivos de configuração e registros de sistema, permanecem disponíveis mesmo após a remoção ou recriação dos contêineres. Além da persistência, os volumes facilitam o compartilhamento de dados entre múltiplos contêineres e oferecem melhor desempenho e segurança em comparação com o armazenamento interno temporário dos contêineres. Essa característica torna os volumes essenciais para aplicações que necessitam armazenar informações de forma permanente.

Volumes no Docker

Os dados armazenados dentro do contêiner são perdidos quando ele é removido.

Para persistência de dados, utiliza-se volumes.

```
docker volume create dados_mysql
```

Montagem

```
docker run -v dados_mysql:/var/lib/mysql mysql
```

Os dados permanecem mesmo após a remoção do contêiner.

Docker Compose

Docker Compose é uma ferramenta utilizada para gerenciar múltiplos contêineres por meio de um único arquivo YAML.

```
</> YAML

services:
  web:
    build: .
    ports:
      - "5000:5000"

  db:
    image: mysql
```

```
docker compose up
```

Permite iniciar aplicações completas com apenas um comando.

Características do Docker

Portabilidade

A aplicação pode ser executada em qualquer ambiente que possua Docker instalado.

Leveza :Não necessita de um sistema operacional completo para cada instância.

Consome menos recursos que máquinas virtuais.

Inicialização Rápida: Contêineres podem ser iniciados em segundos ou até milissegundos.

Escalabilidade: Permite criar múltiplas instâncias de uma aplicação rapidamente.

```
docker compose up --scale web=10
```

Características do Docker

Reprodutibilidade: Todos os ambientes são criados de forma idêntica.

Isolamento: Cada aplicação executa em um ambiente independente.

Problemas em um contêiner não afetam necessariamente os demais.

Automação

- Facilita processos de:
- Integração contínua (CI);
- Entrega contínua (CD);
- Testes automatizados;
- Implantação automática.

Docker versus Máquina Virtual

Característica	Docker	Máquina Virtual
Virtualização	Sistema Operacional	Hardware
Kernel próprio	Não	Sim
Consumo de memória	Baixo	Alto
Tempo de inicialização	Segundos	Minutos
Portabilidade	Alta	Média
Desempenho	Alto	Menor
Tamanho	Pequeno	Grande

Principais Aplicações do Docker

O Docker é amplamente utilizado para:

- Desenvolvimento de aplicações web;
- Microsserviços;
- Computação em nuvem;
- DevOps;
- Integração contínua (CI);
- Entrega contínua (CD);
- Testes automatizados;
- Sistemas distribuídos;
- Inteligência Artificial;
- Ciência de dados;
- Bancos de dados;
- Aplicações corporativas.

Resumo

Em resumo, Docker é uma plataforma de containerização que permite empacotar aplicações e suas dependências em ambientes isolados, leves, portáteis e reproduzíveis. Seu funcionamento baseia-se na criação de imagens e contêineres gerenciados pelo Docker Engine, utilizando recursos do kernel do sistema operacional para fornecer isolamento, eficiência e escalabilidade superiores às abordagens tradicionais baseadas em máquinas virtuais.

